

✓ 1. Judul Praktikum

Praktikum Struktur Data - Topik 7: Linked List (Tugas Praktikum Level Mudah & Menengah)

2. Tujuan Praktikum

- Memvalidasi pemahaman terkait instansiasi objek kelas Node tunggal.
 - Merangkai kerangka Singly Linked List dari nol dan mengisinya dengan agregat data bernilai numerik.
 - Menuntaskan instruksi modifikasi simpul tingkat menengah termasuk penyisipan ekor, pemusnahan elemen spesifik, dan algoritma penghitung panjang rantai.
-

3. Dasar Teori

Manajemen data pada Linked List dikendalikan penuh oleh manipulasi variabel referensi. Pada proses penyisipan di titik awal (Level Mudah), atribut `next` dari simpul baru secara langsung diarahkan ke simpul `HEAD` yang lama, lalu atribut `HEAD` dipindahkan ke simpul baru. Sementara pada perhitungan jumlah elemen atau pencarian (Level Menengah), pola perulangan *while* diwajibkan untuk berjalan menelusuri tiap tautan `next` secara beruntun sampai menemui jalan buntu (terdeteksi nilai `None`).

4. Langkah Praktikum

Seluruh penyelesaian untuk Tantangan 1 hingga 8 disatukan secara rapi dalam satu sel kode di bawah ini. Arsitektur kelas diekspansi untuk memenuhi metode perhitungan jumlah simpul yang diminta pada tingkatan menengah.

```
1 # Definisi Cetak Biru (Blueprint)
2 class Simpul:
3     def __init__(self, data):
4         self.data = data
5         self.lanjut = None
6
7 class SenaraiBerantai:
8     def __init__(self):
9         self.kepala = None
10
11     def sisip_depan(self, data_baru):
12         simpul_baru = Simpul(data_baru)
13         simpul_baru.lanjut = self.kepala
14         self.kepala = simpul_baru
```

```
15
16     def sisip_belakang(self, data_baru):
17         simpul_baru = Simpul(data_baru)
18         if self.kepala is None:
19             self.kepala = simpul_baru
20             return
21         akhir = self.kepala
22         while akhir.lanjut:
23             akhir = akhir.lanjut
24         akhir.lanjut = simpul_baru
25
26     def cabut_simpul(self, kunci):
27         sementara = self.kepala
28         sebelumnya = None
29         if sementara and sementara.data == kunci:
30             self.kepala = sementara.lanjut
31             sementara = None
32             return
33         while sementara and sementara.data != kunci:
34             sebelumnya = sementara
35             sementara = sementara.lanjut
36         if sementara is None:
37             return
38         sebelumnya.lanjut = sementara.lanjut
39         sementara = None
40
41     def lacak_data(self, kunci):
42         sementara = self.kepala
43         indeks = 1
44         while sementara:
45             if sementara.data == kunci:
46                 return True, indeks
47             sementara = sementara.lanjut
48             indeks += 1
49         return False, -1
50
51     def hitung_ukuran(self):
52         jumlah = 0
53         sementara = self.kepala
54         while sementara:
55             jumlah += 1
56             sementara = sementara.lanjut
57         return jumlah
58
59     def baca_rantai(self):
60         sementara = self.kepala
61         daftar_cetak = []
62         while sementara:
63             daftar_cetak.append(str(sementara.data))
64             sementara = sementara.lanjut
65         daftar_cetak.append("END")
66         print(" => ".join(daftar_cetak))
67
68
69 print("===== PENYELESAIAN TINGKAT MUDAH =====")
70
71 print("\n--- Tugas 1: Membuat Node Sederhana ---")
72 node_tunggal = Simpul("Objek Pertama")
73 print(f"Isi data pada simpul terisolasi: {node_tunggal.data}")
74
75 print("\n--- Tugas 2 & 3: Rantai 5 Angka & Proses Traversal ---")
```

```

76 mesin_rantai = SenaraiBerantai()
77 kumpulan_angka = [10, 20, 30, 40, 50]
78 for angka in kumpulan_angka:
79     mesin_rantai.sisip_belakang(angka)
80 print("Hasil perunutan rantai awal:")
81 mesin_rantai.baca_rantai()
82
83 print("\n--- Tugas 4: Penyisipan di Ujung Depan ---")
84 mesin_rantai.sisip_depan(99)
85 print("Sesudah angka 99 dipasang di depan:")
86 mesin_rantai.baca_rantai()
87
88
89 print("\n===== PENYELESAIAN TINGKAT MENENGAH =====")
90
91 print("\n--- Tugas 5: Penyisipan di Ujung Belakang ---")
92 mesin_rantai.sisip_belakang(77)
93 print("Sesudah angka 77 dipasang di paling ekor:")
94 mesin_rantai.baca_rantai()
95
96 print("\n--- Tugas 6: Pencabutan Simpul Spesifik ---")
97 mesin_rantai.cabut_simpul(30)
98 print("Sesudah angka 30 dihancurkan dari tengah:")
99 mesin_rantai.baca_rantai()
100
101 print("\n--- Tugas 7: Pelacakan Data ---")
102 target_cari = 40
103 ditemukan, posisi = mesin_rantai.lacak_data(target_cari)
104 if ditemukan:
105     print(f"Pencarian sukses! Angka {target_cari} terdeteksi di lompatan ke-{posisi}")
106 else:
107     print(f"Pencarian gagal. Angka {target_cari} tidak eksis di dalam sistem.")
108
109 print("\n--- Tugas 8: Perhitungan Total Simpul ---")
110 total_simpul = mesin_rantai.hitung_ukuran()
111 print(f"Setelah seluruh manipulasi, rantai ini sekarang memuat {total_simpul} simpul.")

```

===== PENYELESAIAN TINGKAT MUDAH =====

--- Tugas 1: Membuat Node Sederhana ---

Isi data pada simpul terisolasi: Objek Pertama

--- Tugas 2 & 3: Rantai 5 Angka & Proses Traversal ---

Hasil perunutan rantai awal:

10 => 20 => 30 => 40 => 50 => END

--- Tugas 4: Penyisipan di Ujung Depan ---

Sesudah angka 99 dipasang di depan:

99 => 10 => 20 => 30 => 40 => 50 => END

===== PENYELESAIAN TINGKAT MENENGAH =====

--- Tugas 5: Penyisipan di Ujung Belakang ---

Sesudah angka 77 dipasang di paling ekor:

99 => 10 => 20 => 30 => 40 => 50 => 77 => END

--- Tugas 6: Pencabutan Simpul Spesifik ---

Sesudah angka 30 dihancurkan dari tengah:

99 => 10 => 20 => 40 => 50 => 77 => END

--- Tugas 7: Pelacakan Data ---

Pencarian sukses! Angka 40 terdeteksi di lompatan ke-4.

--- Tugas 8: Perhitungan Total Simpul ---

Setelah seluruh manipulasi, rantai ini sekarang memuat 6 simpul aktif.

5. Analisis Hasil

- Pada penyelesaian tingkat mudah, pembangunan struktur dasar tidak menemukan kendala. Tipe bentukan objek kelas `Simpul` berhasil menampung muatan datanya secara mandiri.
- Pada tingkat menengah, proses perhitungan total simpul (`hitung_ukuran`) membuktikan bahwa program dipaksa untuk melangkah (*traversal*) menembus seluruh gerbong dari ujung ke ujung. Hal ini secara teori memberikan bukti konkret bahwa kompleksitas kalkulasi rentang/ukuran pada Singly Linked List adalah mutlak bernilai linear, tidak seinstan Array yang sudah mendaftarkan ukurannya di awal.

6. Jawaban Diskusi

1. **Definisi Konseptual:** Sebuah fondasi struktur linear di mana elemen-elemennya ditanam di titik-titik memori yang tersebar bebas, namun dirangkai menjadi kesatuan utuh menggunakan bantuan pengait penunjuk alamat (*pointer*).
2. **Alasan Disebut Dinamis:** Karena batas tampungannya tidak perlu dikunci di awal. Rantai ini bisa tumbuh memanjang saat ditambahkan data baru atau menyusut saat ada data yang dibuang tepat pada saat kode sedang berjalan (*runtime*).
3. **Kontras dengan Array:** Array mencetak kapling tanah memori secara berjejer rapat, unggul di pembacaan langsung pakai nomor rumah (indeks). Senarai berantai menempati kapling tanah yang acak-acakan, unggul karena bongkar-pasang penghuninya tidak mengganggu tetangga lain.
4. **Peran Vital Pointer:** Menjadi satu-satunya kompas atau tali penuntun yang memberitahu komputer di mana lokasi penyimpanan bongkahan data berikutnya. Tanpanya, data tersebut akan hilang di lautan memori tak bertepi.
5. **Varian Singly vs Doubly:** Varian *Singly* ibarat jalan satu arah; hanya bisa bergerak maju ke data selanjutnya. Varian *Doubly* melengkapinya dengan spion; memiliki dua pengait sehingga bisa menengok ke data sebelumnya (*prev*) dan maju ke data selanjutnya (*next*).
6. **Keuntungan Mode Melingkar (Circular):** Ekor barisan diikat kembali ke kepala barisan. Sangat krusial untuk program yang harus terus berputar tanpa henti,

seperti sistem gilir pembagian jatah CPU pada sistem operasi atau rotasi pemain pada papan permainan bergilir.

7. **Penyebab Kelambanan Akses:** Komputer tidak bisa menebak lokasi elemen ke-50. Ia harus mengetuk pintu dari elemen ke-1, menanyakan alamat ke-2, lalu ke-3, hingga akhirnya tiba di tujuan.
8. **Momen Terbaik Penggunaan:** Sangat direkomendasikan ketika intensitas program didominasi oleh operasi bongkar-pasang, tambal-sulam, atau selip data secara ekstrem di tengah antrean yang terus-menerus berubah ukurannya.
9. **Contoh Adaptasi Komputasi Terkini:**
 - Alur maju-mundur pada peramban situs web (*Browser History*).
 - Alur bolak-balik pengeditan kanvas atau dokumen (*Undo & Redo*).
 - Rantai antrean berkas sinkronisasi sistem awan.
10. **Batu Loncatan Struktur Lain:** Ia adalah struktur lego dasar paling fleksibel yang pada akhirnya diadaptasi untuk membangun ruang memori dinamis untuk tumpukan (Stack), antrean peladen (Queue), hingga cabang jaringan beranting pada hierarki Graf (Graph) dan Pohon (Tree).